

MotorLógico

Arquitectura Técnica del Sistema CSP

El **MotorLógico** (MotorPascal) es un sistema de resolución de Problemas de Satisfacción de Restricciones (**CSP**) que combina Free Pascal como capa de orquestación con la librería C++ **Gecode** como motor de búsqueda. La comunicación entre capas usa un bridge con convención C (*extern "C"*), garantizando compatibilidad binaria sin recompilación en runtime.

1. Componentes del Sistema

El sistema se compone de cuatro piezas fundamentales que trabajan de forma coordinada. Cada componente tiene una responsabilidad clara y bien delimitada:

Componente	Tipo	Responsabilidad
<code>TestGecodeBridge.pas</code>	Pascal — Orquestador	Punto de entrada. Lee argumentos, invoca el parser JSON, llama <code>csp_create/add/solve</code> , imprime resultados.
<code>UCSPJson.pas</code>	Pascal — Parser	Parsea el JSON con AST (<code>Json_output</code>). Produce <code>TCSPData</code> con arrays de <code>TCSPVar</code> y <code>TCSPConstraint</code> .
<code>UGecodeBridge.pas</code>	Pascal — Bridge API	Define tipos compartidos, helpers (<code>CSPMakeVar</code> , <code>CSPLinear</code>), y declara las funciones <code>csp_*</code> como <code>cdecl; external</code> .
<code>gecode_bridge.o</code>	C++ precompilado	Implementa <code>CSPModel.Space</code> . Recibe variables y constraints, construye propagadores Gecode, ejecuta DFS con backtracking.

■ **Clave del linking:** La directiva `{$L gecode_bridge.o}` en `UGecodeBridge.pas` le indica al compilador FPC que incluya el objeto C++ al linkear. Las declaraciones `cdecl; external` resuelven los símbolos en tiempo de build.

2. Pipeline de Ejecución (7 Pasos)

El pipeline transforma una expresión de constraint en texto libre hasta una solución entregada de vuelta a Pascal. Dos archivos JSON distintos marcan la frontera entre parsing y resolución:

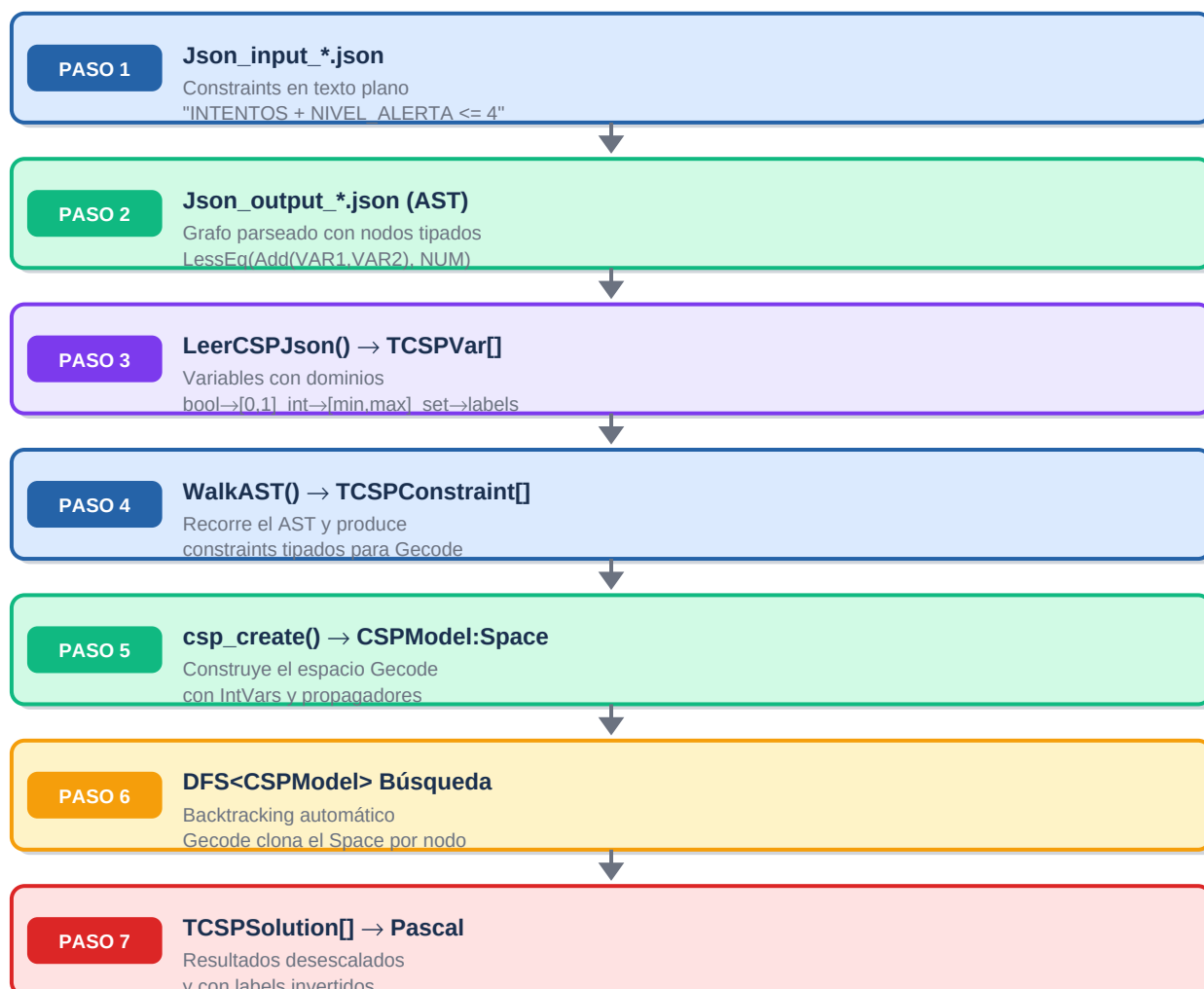


Figura 1 — Pipeline completo de 7 pasos

Dos JSON distintos: `Json_input_*.json` contiene las constraints como texto plano legible por humanos. `Json_output_*.json` contiene el AST ya parseado. La herramienta externa **JsonToGraph** hace el parsing de texto a grafo. **Pascal solo lee el output (AST)**, nunca procesa texto de constraint directamente.

3. Representación AST como Grafo

Cada constraint en el JSON-output es un árbol sintáctico abstracto (AST). Los nodos son operadores predefinidos con tipos fijos. **WalkAST()** recorre el árbol y produce el **TCSPConstraint** correspondiente para Gecode:

Ejemplo: "INTENTOS + NIVEL_ALERTA <= 4"

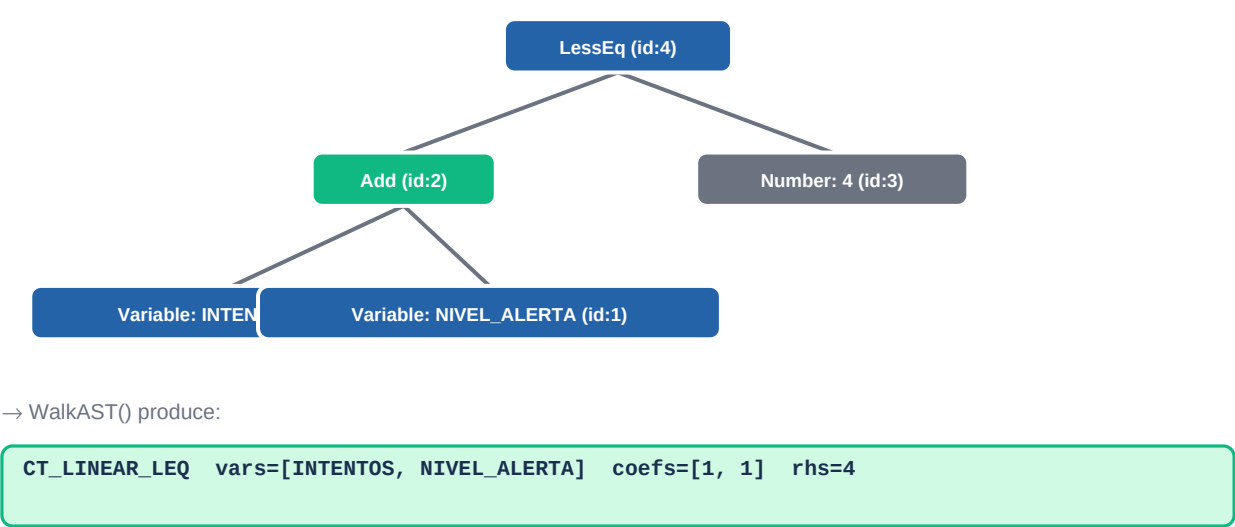


Figura 2 — AST para "INTENTOS + NIVEL_ALERTA ≤ 4" y su traducción

Tipos de Nodo Disponibles

Tipo	Código	Descripción
Hojas	NT_VARIABLE (0), NT_NUMBER (1)	Nombre de variable o valor numérico literal
Comparación	NT_EQUALS (2), NT_NOTEQUALS (3) NT_LESS (4), NT_GREATER (5) NT_LESSEQ (6), NT_GREATEREQ (7)	Operadores relacionales == <> < > <= >=
Lógicos	NT_AND (8)	Divide en dos constraints independientes. NT_OR no soportado
Aritméticos	NT_ADD (11), NT_SUBTRACT (12) NT_MULTIPLY (13), NT_NEGATE (15)	Para expresiones lineales. Multiply solo const×var
Especiales	NT_FUNCCALL (16) abs(), dist()	Funciones built-in para valor absoluto y distancia
Conjuntos	NT_IN (17), NT_SET (18) NT_INTERVAL (19)	Pertenencia a conjunto {a,b,c} o intervalo [lc,hi]

4. El Bridge Pascal ↔ C++

El bridge es la pieza central de la arquitectura. Usa la convención C (extern "C") para exponer 7 funciones públicas desde C++. Todo lo demás — la clase CSPModel, los propagadores, el mecanismo de copia — permanece oculto bajo #pragma GCC visibility(hidden):

Función C	Firma Pascal	Acción
-----------	--------------	--------

csp_create	function csp_create(vars:PCSPVar; n:Integer): Pointer; cdecl; external;	Instancia CSPModel con n variables y sus dominios
csp_add_constraint	function csp_add_constraint(m:Pointer; c:PCSPConstraint): Integer; cdecl; external;	Agrega un propagador al Space de Gecode
csp_solve_first	function csp_solve_first(m:Pointer; sol:PCSPSolution): Integer; cdecl; external;	Busca la primera solución con DFS
csp_solve_all	function csp_solve_all(m:Pointer; callback:...): Integer; cdecl; external;	Enumera todas las soluciones con callback
csp_free	procedure csp_free(m:Pointer); cdecl; external;	Libera la memoria del CSPModel

■ **Contrato de copia con Gecode:** Durante la búsqueda DFS, Gecode necesita clonar el Space en cada nodo del árbol para hacer backtracking. La clase CSPModel implementa `Space* copy()` que retorna `new CSPModel(*this)`. El copy constructor actualiza cada `IntVar` al nuevo espacio con `v.update(*this, src.v)`. Sin esto, el backtracking no funciona.

5. Tipos de Variables y Dominios

UCSPJson.pas convierte cada variable del JSON a una `IntVar` de Gecode según su tipo declarado. Los tipos numéricos usan un factor de escala $\times 1000$ para representar decimales como enteros:

Tipo JSON	Dominio IntVar	Notas
boolean	IntVar [0, 1]	0 = false, 1 = true
integer	IntVar [min, max]	Rangos definidos en el JSON
numeric	IntVar [min $\times 1000$, max $\times 1000$]	Escala $\times 1000$. Al retornar: $\div 1000$
set	IntVar [0, N-1]	Label map: NORTE=0, SUR=1... Se invierte al retornar
value presente	CT_IN_SET inmediato	Variable fijada a valor específico en la creación

6. Contexto Arquitectónico

Los patrones usados son sólidos y establecidos en la industria. La combinación específica es inusual, pero cada pieza sigue prácticas estándar:

Componente	¿Es usual hoy?	Alternativa moderna equivalente
Bridge via extern "C"	✓ Muy común	Python ctypes, Rust FFI, Go cgo — mismo patrón
Structs planos como ABI	✓ Estándar	Es el único ABI portable entre lenguajes
AST como grafo con tipos fijos	✓ Común en compiladores	LLVM IR, query plans de BD, expression trees
Pascal como host language	■ Inusual hoy	Python, Java, o directamente C++ serían más comunes
Gecode sin MiniZinc	■ Poco usual	MiniZinc → Gecode es el flujo estándar
Parser JSON manual	✗ Muy inusual	json.loads(), serde_json, nlohmann/json

■ ■ **Analogía en producción:** El patrón más cercano en producción es TensorFlow/PyTorch: Python como API de alto nivel → pybind11/ctypes → C++ core con operaciones → CUDA/MKL. La idea es idéntica: un lenguaje cómodo como interfaz, con el cómputo pesado en C/C++ precompilado.

7. Persistencia: JsonSink + JsonSource + SQLite

El pipeline usa **SQLite como memoria entre etapas**. El binding sigue el mismo patrón que con Gecode: llamadas directas a la C API de libsqlite3.so, sin ORM, sin librería intermedia, sin nada:

```
function sqlite3_open(...) cdecl; external 'sqlite3'; function sqlite3_step(...) cdecl; external 'sqlite3';
```

libsqlite3.so está presente en cualquier Linux. Un solo .so de ~1 MB. El resultado: **cero servidores, cero daemons, cero configuración**. El deployment es copiar los binarios.

Flujo del pipeline con persistencia:

```
# Guardar resultado de una etapa ./JsonToGraph sistema.json | ./JsonSink pipeline.db
graph # Procesar y guardar cada rama ./JsonSource pipeline.db graph | ./FwdConsistency |
./JsonSink pipeline.db fwd ./JsonSource pipeline.db graph | ./BwdConsistency | ./JsonSink
pipeline.db bwd # Continuar desde cualquier punto ./JsonSource pipeline.db fwd |
./TestGecodeBridge
```

La tabla runs como log del pipeline

```
CREATE TABLE runs ( id INTEGER PRIMARY KEY AUTOINCREMENT, ts TEXT DEFAULT
(strftime('%Y-%m-%dT%H:%M:%f','now')), tag TEXT, -- 'graph', 'fwd', 'bwd', 'csp'...
status TEXT, -- 'arc_consistent', ... payload TEXT -- el JSON completo );
```

Cada etapa guarda su JSON completo. Eso da gratis:

Beneficio	Descripción
Replay	Relanzar cualquier etapa desde un punto anterior sin recalcular todo
Debug	Inspeccionar el estado exacto en cada nodo del pipeline
Auditoría	Historial completo con timestamp ISO de cada decisión
Bifurcación	FwdConsistency y BwdConsistency pueden correr en paralelo desde el mismo tag

■ **El stack completo sin instalar nada:**

Binarios Pascal (~250 KB c/u, estáticos, sin deps) → stdin/stdout Unix pipes → libsqlite3.so (presente en cualquier Linux) → un archivo pipeline.db portable. El 'deployment' es **copiar los binarios**.

8. El Sistema como Base para un Agente/Bot

La arquitectura es especialmente apta para construir **agentes que toman decisiones bajo restricciones formales**. Cada componente cubre una necesidad concreta del agente:

Necesidad del agente	Qué lo cubre	Cómo
Memoria persistente	SQLite + tabla runs	Cada decisión guardada con timestamp. JsonSource recupera
Razonamiento formal	CSP solver (Gecode)	Dado el estado del mundo como variables+constraints, respondo
Composabilidad	Unix pipes + binarios	Cada etapa es un proceso independiente. El agente puede enc
Interfaz en lenguaje natural	LLM como frontend	El LLM traduce lenguaje natural a Json_input. El CSP garantiza
Auditabilidad	Historial SQLite	Todo el historial de razonamiento queda trazable con tags y tim
Portabilidad / Edge	Binarios estáticos	Sin servidor, sin cloud, sin configuración. Funciona en edge o s

Integración LLM + CSP Solver

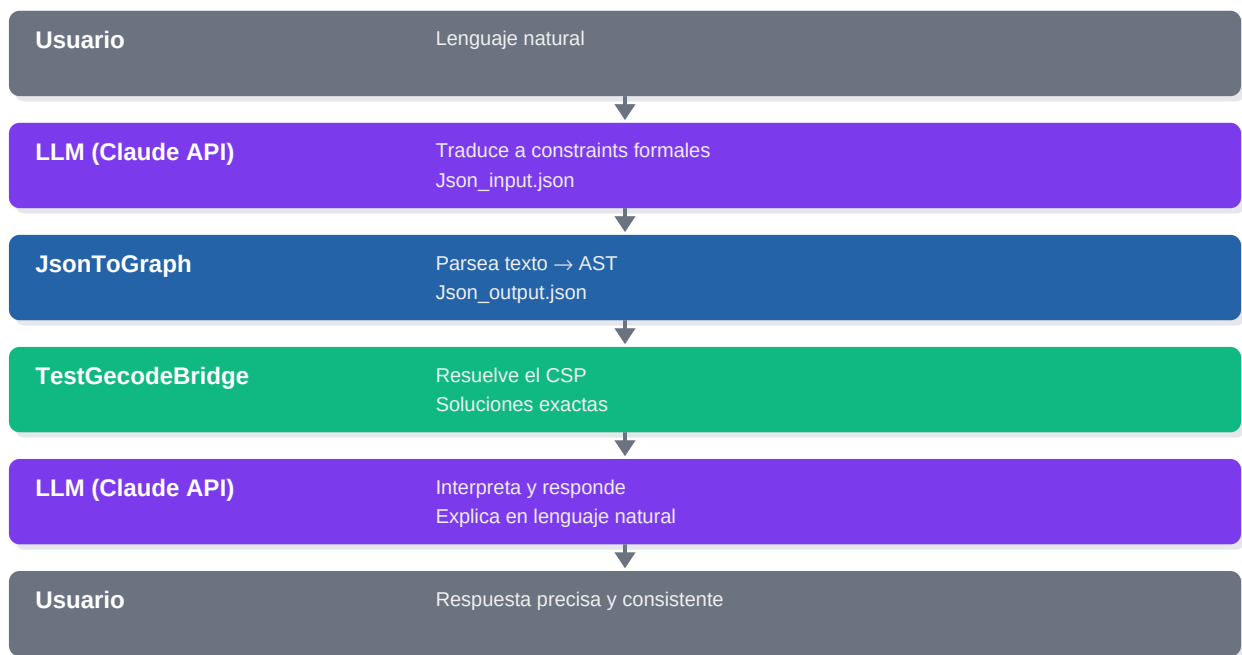


Figura 3 — Integración LLM + MotorLógico para agente conversacional

■ **Por qué funciona la combinación LLM + CSP:** El LLM es probabilístico y creativo pero no garantiza consistencia lógica. El CSP solver es exacto: si hay solución la encuentra, si no la hay lo dice definitivamente. La combinación produce un agente **flexible en la entrada** (LLM traduce lenguaje natural a constraints), **exacto en el razonamiento** (Gecode verifica consistencia formal) y **auditable** (todo el historial en SQLite). Casos de uso naturales: scheduling, configuración, asignación de recursos, control de acceso.

9. El Binario es Genérico — El .db es el Bot

Este es el insight más poderoso del diseño: **TestGecodeBridge no sabe nada del dominio**. El ejecutable es completamente genérico. Lo que define la misión de cada bot es su base de datos:

Base de datos	Variables del dominio	Misión
agent_acceso.db	ZONA, PERFIL, HORARIO, ALARMA	Control de acceso físico a edificios
agent_scheduling.db	TAREA, RECURSO, TIEMPO, PRIORIDAD	Planificación y asignación de tareas
agent_inventario.db	PRODUCTO, STOCK, PROVEEDOR, UMBRAL	Gestión de inventario y reposición
agent_medico.db	SINTOMA, MEDICAMENTO, DOSIS, CONDICION	Asistencia en decisiones clínicas bajo restricciones

Mismo binario. Distinta misión.

La Misión está Codificada en los Constraints

knowledge base = variables (qué existe en mi mundo)
+ constraints (qué reglas rigen mi mundo)
+ runs (qué ha pasado – mi memoria)

Los constraints son el conocimiento del dominio. Un bot de control de acceso sabe que `ALARMA_ACTIVADA = true` → `PUERTA_ABIERTA = false` porque está en su modelo. **No porque le enseñaron con ejemplos** — porque está definido formalmente. Esto garantiza consistencia verificable, no estadística.

Deploy de un Nuevo Bot

```
# Crear un nuevo agente con su perfil cp template.db agent_nuevo.db # Cargar su
knowledge base ./JsonToGraph mision_nueva.json | ./JsonSink agent_nuevo.db knowledge #
El agente ya está listo ./JsonSource agent_nuevo.db knowledge | ./TestGecodeBridge
```

Multi-Agente Natural

Como cada bot es un `.db` independiente, corren en paralelo sin interferencia. SQLite maneja concurrencia de lectura sin problema:

```
./JsonSource acceso.db knowledge | ./TestGecodeBridge & ./JsonSource scheduling.db
knowledge | ./TestGecodeBridge & ./JsonSource inventario.db knowledge |
./TestGecodeBridge &
```

Separación de Responsabilidades: la Clave Filosófica

La mayoría de los agentes LLM mezclan conocimiento, razonamiento y memoria en el modelo. En el MotorLógico están separados limpiamente:

Qué	Dónde vive	Cambia cuándo
Conocimiento del dominio	constraints en el <code>.db</code>	Se cambia el <code>.db</code> — el binario no toca
Motor de razonamiento	TestGecodeBridge (genérico)	Solo si se mejora el motor core — nunca por do
Memoria episódica	tabla runs en el <code>.db</code>	Crece con cada ejecución — historial persistente
Interfaz natural	LLM (externo, opcional)	Reemplazable — el core no depende de él

■ **Cambiar la misión del bot es cambiar el .db. El resto no toca.**

Este principio de separación — conocimiento declarativo en datos, razonamiento en código genérico — es lo que hace al sistema escalable, mantenible y auditable. Cada bot puede evolucionar su knowledge base de forma independiente, sin recompilar, sin reconfigurar, sin tocar el motor que garantiza la consistencia formal.